

LAMDA & API GATEWAY

AWS Lambda – Benefits

No server management – AWS handles provisioning and scaling automatically.

1. Pay only for execution time, making it highly cost-effective.
 2. Automatically scales based on incoming events or requests.
 3. Supports event-driven architecture with services like S3, API Gateway, and EventBridge.
 4. Built-in integration with CloudWatch for logging and monitoring.
-

Amazon API Gateway – Benefits

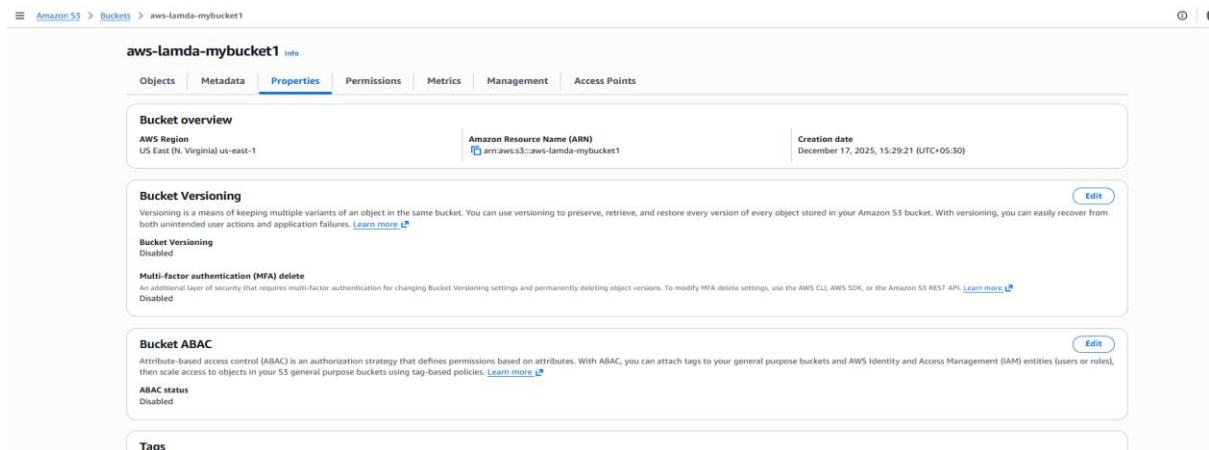
1. Easily exposes backend services as secure REST or HTTP APIs.
2. Integrates seamlessly with AWS Lambda for serverless APIs.
3. Provides built-in authentication, authorization, and throttling.
4. Supports caching to improve performance and reduce backend load.
5. Automatically scales to handle high traffic without manual intervention.

1. **Lambda Task 1: S3 File Upload Processing Goal: Trigger Lambda when a file is uploaded to S3. What to do 1. Create S3 bucket 2. Create Lambda (Python) 3. Add S3 PUT trigger 4. Print file name & size in CloudWatch logs You will learn • Lambda triggers • IAM role for Lambda • CloudWatch logs**

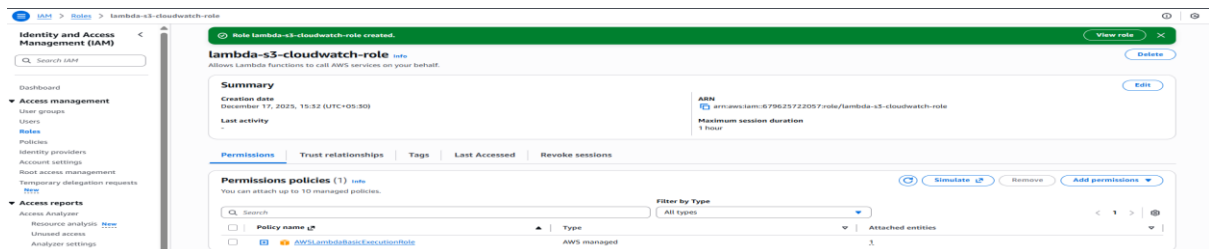
Steps:

- Created an S3 bucket to store uploaded files.
- Created an IAM role with CloudWatch logging permissions for Lambda.
- Created a Lambda function using Python runtime.
- Attached the IAM role to the Lambda function.
- Configured the S3 bucket to trigger Lambda on PUT (upload) events.
- Wrote Lambda code to read S3 event metadata.
- Extracted bucket name, file name, and file size from the event.
- Printed file details using print() statements.
- Uploaded a file to the S3 bucket to test the trigger.
- Verified execution output in Amazon CloudWatch Logs.

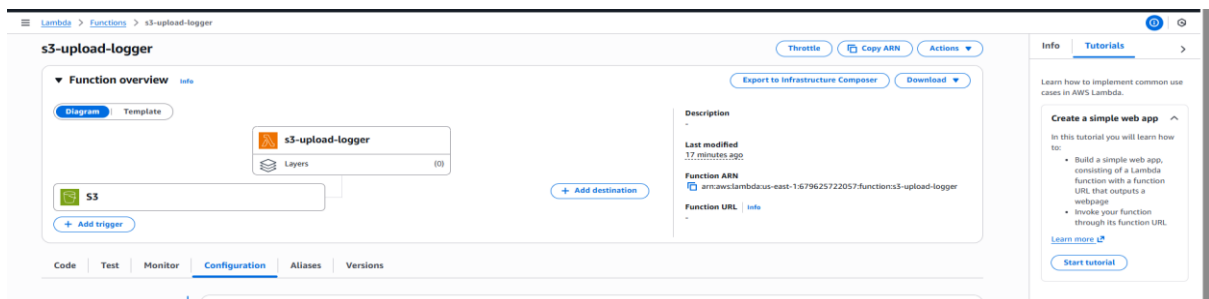
I have created s3 bucket



I have created a new role in IAM under role section 'AWS service role' AWS basic execution permission provided by aws policies



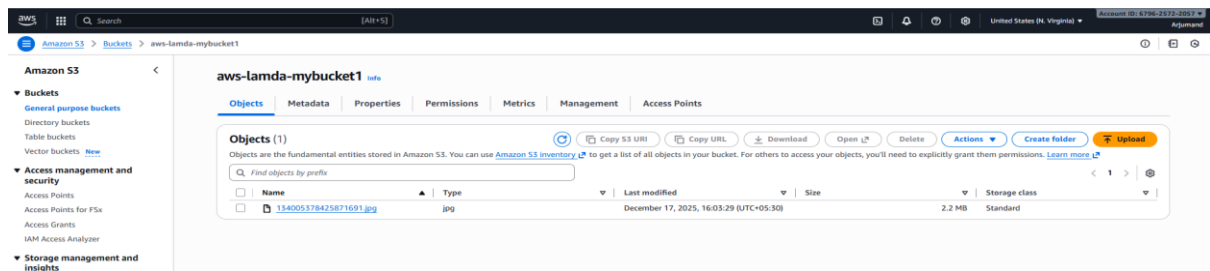
I have created a lamda function, provided name and selected python3.12 wil all default



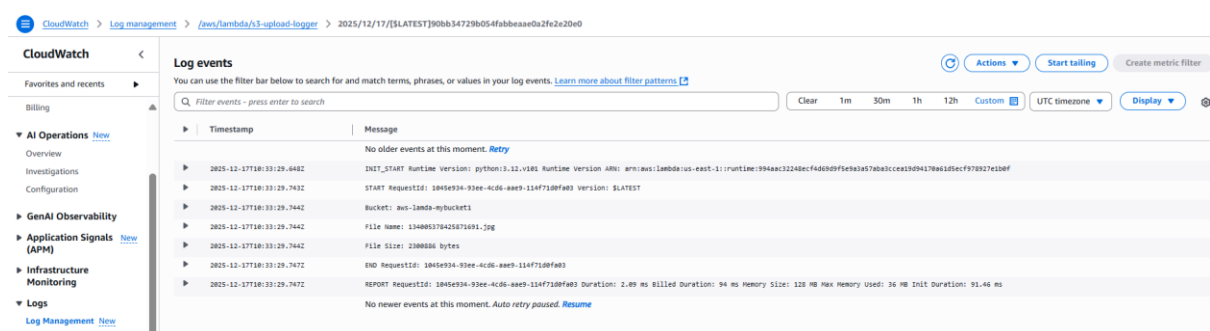
Wrote a python code in code section to run automatically s3 activities in cloud watch using lamda.



Added file in s3 bucket to check the activity in cloud watch and to verify whether code is running properly or not



Verified by checking cloud watch it is capturing logs using lambda fuction and python code is also accurate.



2. Lambda Task 2: EC2 Auto Start / Stop (Cost Optimization)

Goal: Automatically stop/start EC2.

What to do

1. **Create Lambda (Python + boto3)**
2. **Write code to stop EC2**

Steps:

Created IAM role for Lambda

Created Lambda function (Python + boto3)

Used EC2 API to:

- Start instance
 - Stop instance
- Tested manually from Lambda console
EC2 state changes successfully

I have created a role in which I have given two permissions

lambda-ec2-start-stop-role-role-h13e99nq [info](#) [Delete](#) [Edit](#)

Summary

Creation date
December 17, 2025, 17:05 (UTC+05:30)

Last activity
31 minutes ago

ARN
arn:aws:iam::679625722057:role/service-role/lambda-ec2-start-stop-role-role-h13e99nq

Maximum session duration
1 hour

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) [Info](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Filter by Type
All types

Policy name	Type	Attached entities
AmazonEC2FullAccess	AWS managed	2
AWSLambdaBasicExecutionRole-7c8f4cd1-86bc-4280-8199-f665...	Customer managed	1

► **Permissions boundary** (not set)

I have created a function in lamda withr python 3.10

lambda-ec2-start-stop-role [Throttle](#) [Copy ARN](#) [Actions](#)

Function overview [info](#)

[Diagram](#) [Template](#)

[+ Add trigger](#) [+ Add destination](#)

Description

Last modified
43 minutes ago

Function ARN
arn:aws:lambda:us-east-1:679625722057:function:lambda-ec2-start-stop-role

Function URL [info](#)

Test event [info](#) [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action
[Create new event](#) [Edit saved event](#)

Invocation type

I have given python script to start and stop instance

Code source [info](#) [Open in Visual Studio Code](#) [Upload from](#)

```
1 import boto3
2
3 ec2 = boto3.client('ec2')
4
5 INSTANCE_IDS = ['i-0b2b7eb2c5f0970e0'] # Replace with your EC2 instance ID
6
7 def lambda_handler(event, context):
8     action = event.get('action')
9
10    if action == 'start':
11        ec2.start_instances(InstanceIds=INSTANCE_IDS)
12        print("EC2 instance started")
13    elif action == 'stop':
14        ec2.stop_instances(InstanceIds=INSTANCE_IDS)
15        print("EC2 instance stopped")
16    else:
17        print("No valid action provided")
18
19
20
```

Test event [info](#) [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action
[Create new event](#) [Edit saved event](#)

Invocation type
☒ Synchronous
☐ Asynchronous

Event name
MyEventName

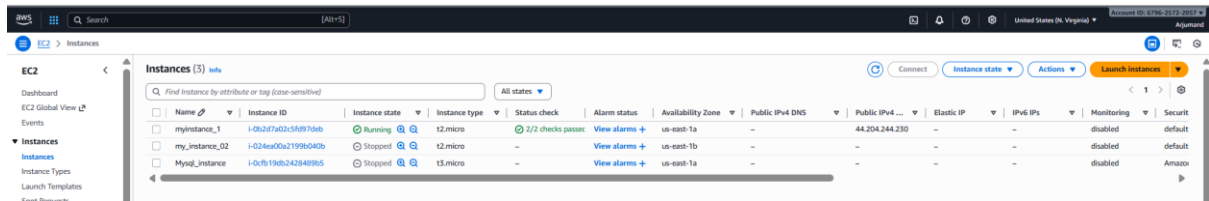
Event sharing settings
☒ Private
☐ Shareable

Template - optional
Hello World

Event JSON

```
1 {
2   "action": "start"
3 }
4
```

When I am testing with start action it is starting the instance and when I am testing with stop action it is stopping the instance.



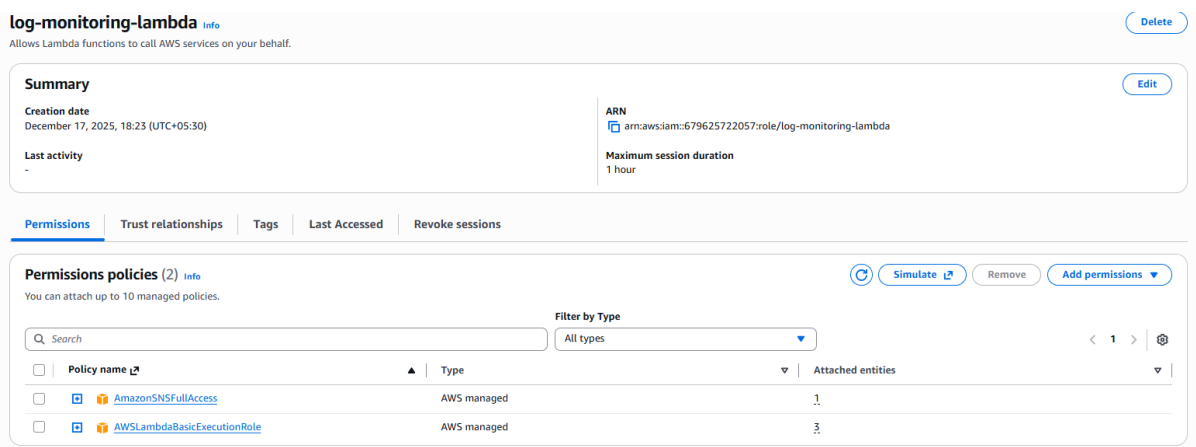
3. Lambda Task 3: Log Monitoring & Alert

Goal: Send alert when ERROR appears in logs.

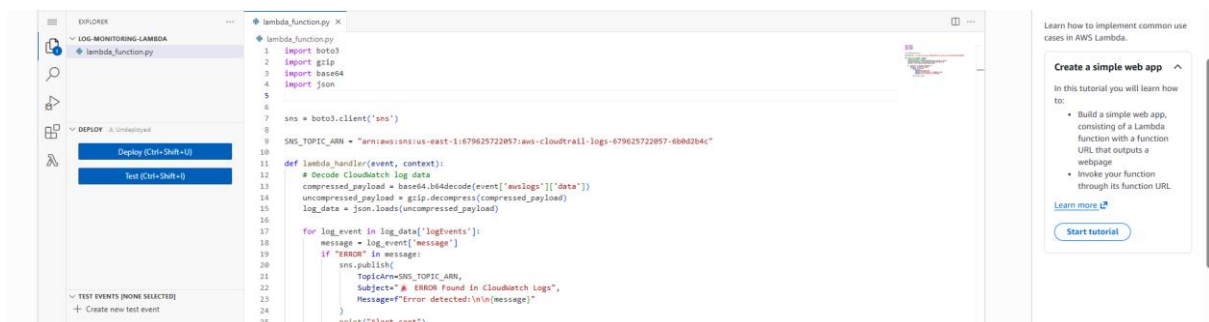
What to do

1. Create Lambda
2. Subscribe Lambda to CloudWatch Logs
3. If log contains ERROR → send SNS email

Created lambda and given permission sns full access and aws lambda basic execution role



Provided lambda to python script for monitoring alert



Created lambda subscription filter in cloudwatch under subscription filter

The screenshot shows the AWS Lambda console interface for the 'lambdamonitoring' log group. The 'Subscription filters' tab is selected, displaying a table with one filter named 'ERROR'. The filter pattern is 'ERROR', the destination ARN is 'arn:aws:lambda:us-east-1:679625722057:log-group:lambdamonitoring:', and the subscription filter type is 'Log group-level'. The table also shows columns for 'Applied on transformed logs', 'Field selection criteria', and 'Emit system fields', all of which are currently empty.

Created an other lambda function as you see

The screenshot shows the AWS Lambda console interface for the 'log-test-source' function. The 'Function overview' tab is selected, displaying a diagram of the function's architecture. The function is triggered by 'CloudWatch Logs' and has a 'log-test-source' layer. The 'Add destination' button is visible, and the 'Function overview' section shows the function's description, last modified time (9 minutes ago), and function ARN.

Given a python fuction to print error which is same we have given in cloudwatch subscription

The screenshot shows the AWS Lambda console interface for the 'log-test-source' function. The 'Code source' tab is selected, displaying the function's code. The code is a Python function named 'lambda_handler' that takes 'event' and 'context' as arguments and prints 'ERROR Test error log from source lambda'. The 'Code source' section also shows the function's code editor and the 'Add destination' button.

Deployed and tested the python script in lambda

The screenshot shows the AWS Lambda console interface for the 'log-test-source' function. The 'Test' tab is selected, displaying the function's test results. The 'Test event' section shows the function's test event, and the 'Test' button is visible. The 'Test' section also shows the function's test event and the 'Test' button.

Error was recorded after executing the python script which is same we will get the email we have enabled alert by sns

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Q Filter events - press enter to search

Clear

1m

30m

1h

12h

Custom

UTC timezone

Display

▶ Timestamp

Message

No older events at this moment. [Retry](#)

▶ 2025-12-17T13:53:43.689Z

INIT_START Runtime Version: python3.14.v32 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:lee4e6d61a5efb29083b07572cc670e92de4538be7b21838aed09675084

▶ 2025-12-17T13:53:43.818Z

START RequestId: 3360e5a5-dec1-4ee7-b80c-023a59c5a4c5 Version: SLATEST

▶ 2025-12-17T13:53:43.811Z

ERROR Test error log from source lambda

▶ 2025-12-17T13:53:43.827Z

END RequestId: 3360e5a5-dec1-4ee7-b80c-023a59c5a4c5

▶ 2025-12-17T13:53:43.827Z

REPORT RequestId: 3360e5a5-dec1-4ee7-b80c-023a59c5a4c5 Duration: 15.63 ms Billed Duration: 134 ms Memory Size: 128 MB Max Memory Used: 40 MB Init Duration: 117.39 ms

▶ 2025-12-17T13:56:46.056Z

START RequestId: b32be086-eb42-4788-9da4-d82c8d9ec08c Version: SLATEST

▶ 2025-12-17T13:56:46.064Z

ERROR Test error log from source lambda

▶ 2025-12-17T13:56:46.085Z

END RequestId: b32be086-eb42-4788-9da4-d82c8d9ec08c

▶ 2025-12-17T13:56:46.085Z

REPORT RequestId: b32be086-eb42-4788-9da4-d82c8d9ec08c Duration: 9.72 ms Billed Duration: 10 ms Memory Size: 128 MB Max Memory Used: 40 MB

No newer events at this moment. [Auto retry paused](#). [Resume](#)

4. Lambda Task 4: Delete Backups When EC2 Is Terminated

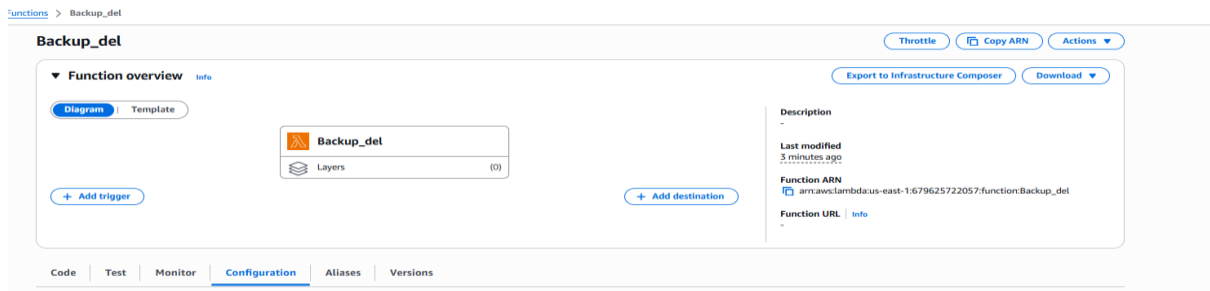
Goal: Auto-delete EBS snapshots after instance termination.

What to do

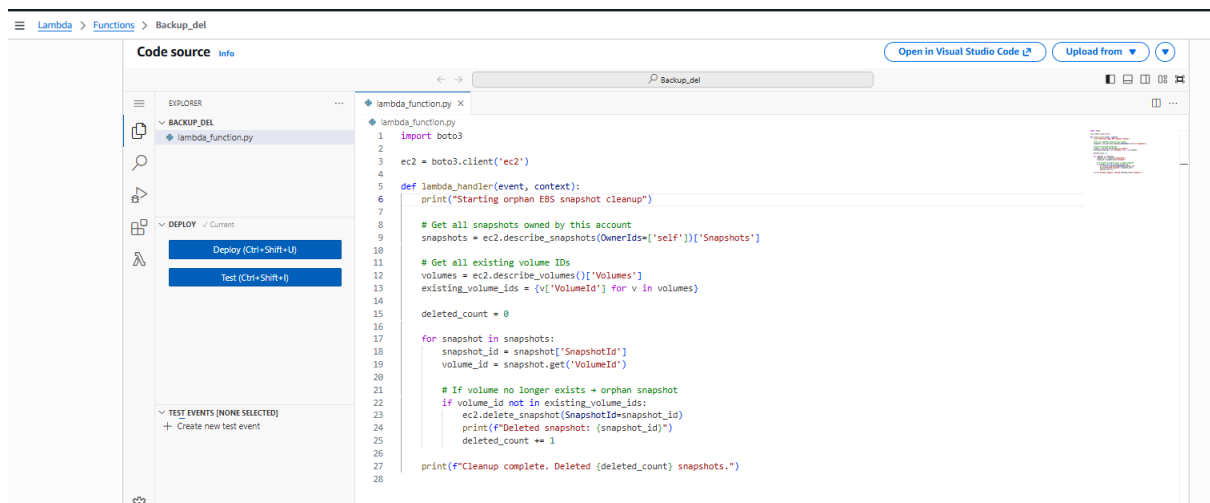
1. Create EventBridge rule for EC2 terminated
2. Trigger Lambda
3. Find EBS snapshots
4. Delete snapshots only if tag = AutoDelete:true

I have created one lambda function to delete unwanted backups for cost optimisation and attach role to it

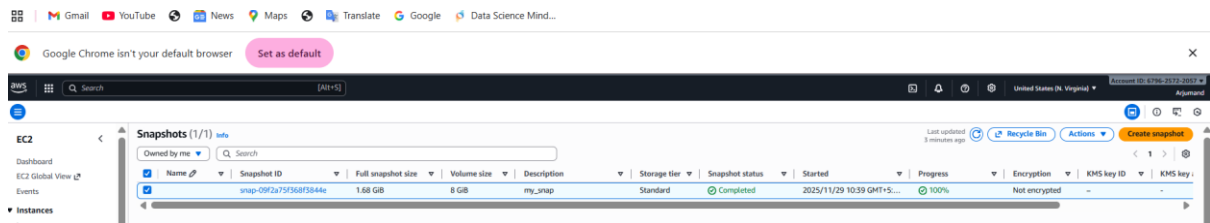
The screenshot displays two parts of the AWS Management Console. The top part shows the 'Volumes' page in the EC2 console, listing a single volume 'vol-0c253ecfba1105e1b' of type 'gp3' with a size of 8 GiB. A green notification banner at the top states 'Successfully created snapshot snap-020bf42c7796222ba0 from volume vol-0c253ecfba1105e1b'. The bottom part shows the 'Permissions' page in the IAM console for a role named 'lambda-ec2-start-stop-role-h33d9nq'. It displays the 'Permissions policies' section, which includes 'AmazonEC2FullAccess' and 'AWSLambdaBasicExecutionRole'.



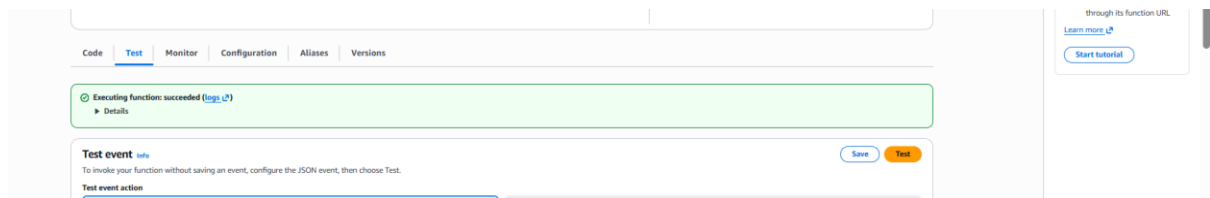
After creating lambda function I have given python script to delete backup whose volume is deleted.



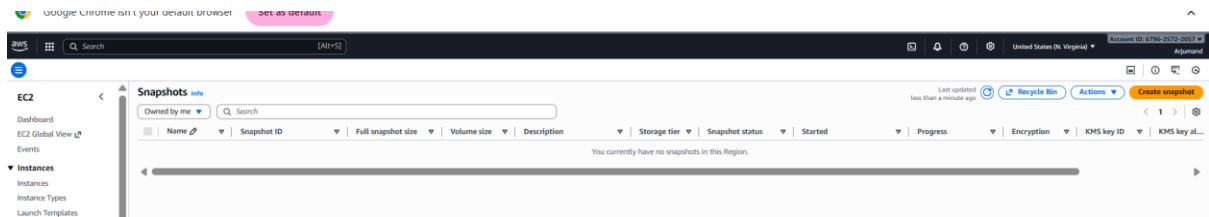
You can see I have backup I am going to deploy and test my python script in lambda fuction to delete snapshot



I deployed my python script and test it



Backup is now deleted successfully.



5. API Gateway Task 1: Create Lambda-Backed API

Goal: Expose Lambda as REST API.

What to do

1. **Create HTTP API**
2. **Create route:**
 - **GET /health**
3. **Integrate with Lambda**
4. **Test using browser / Postman**

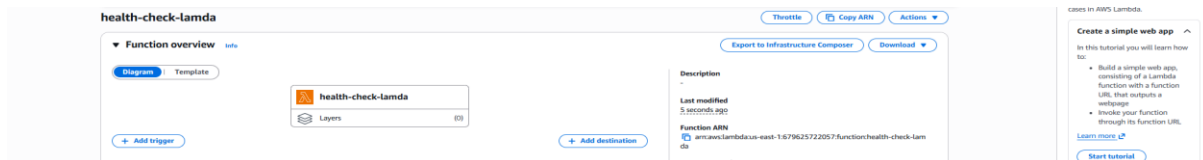
You will learn

- **API → Lambda integration**
- **HTTP methods**

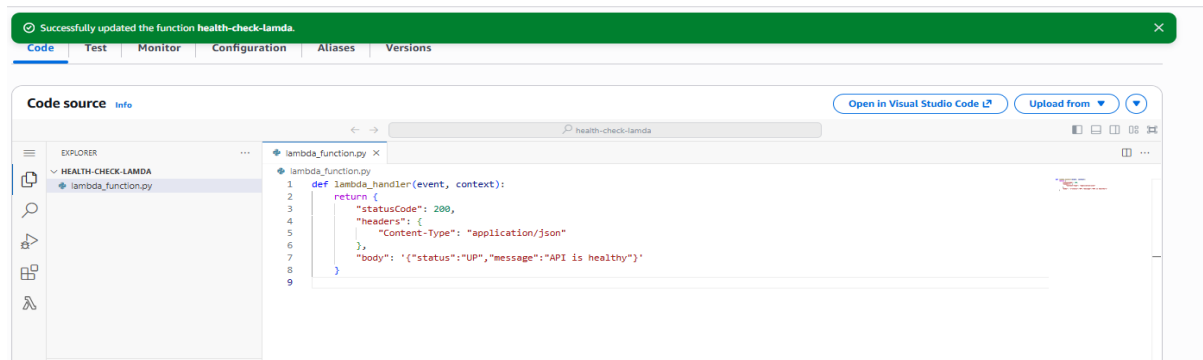
Steps:

- Created a Lambda function using Python to return a health status response.
- Opened API Gateway and created a new HTTP API.
- Integrated the HTTP API with the Lambda function.
- Created a GET route with the path /health.
- Deployed the API to the \$default stage.
- Copied the Invoke URL from the Stages section.
- Tested the API in a browser/Postman using `https://<api-id>.execute-api.<region>.amazonaws.com/health`.
- Verified the response and checked CloudWatch logs to confirm Lambda execution.

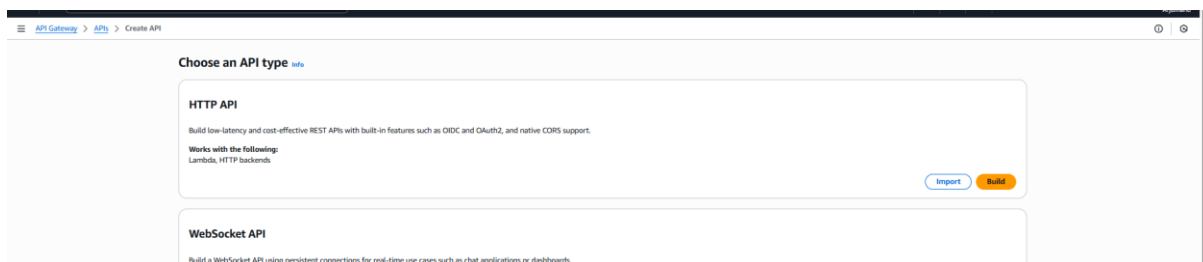
I have created one lambda function I took python and selected basic lambda permissions.



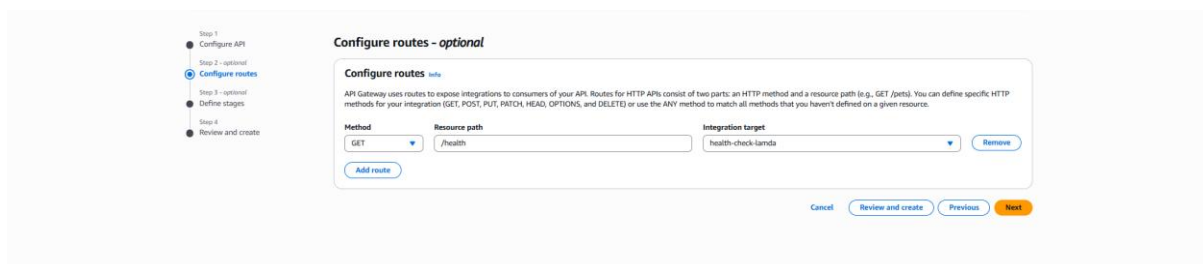
I have provided python script for health check in code tab



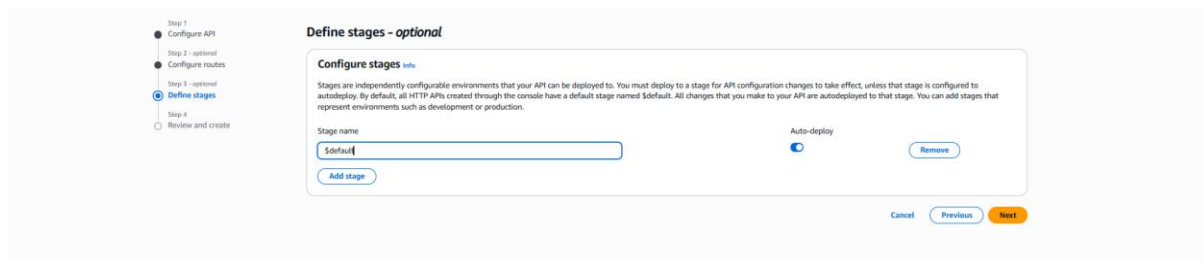
I have created one API with HTTP API



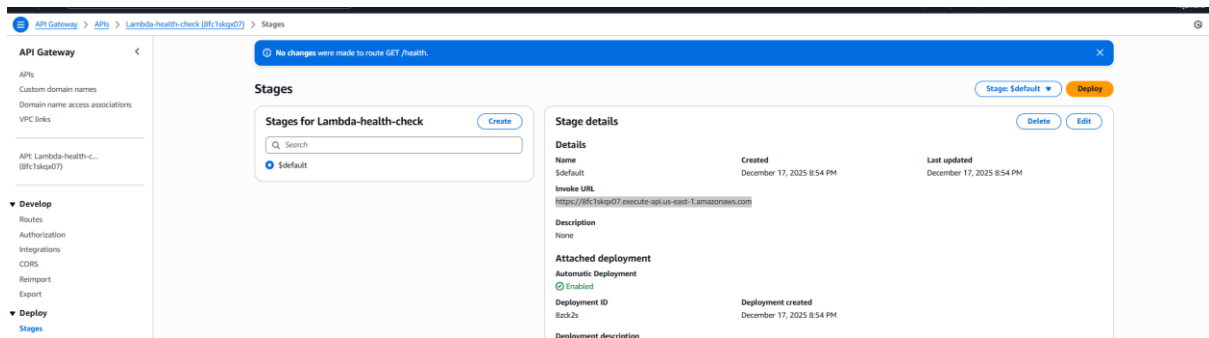
I selected get in method /health in resource and integration target will be my lambda fuction which I have created



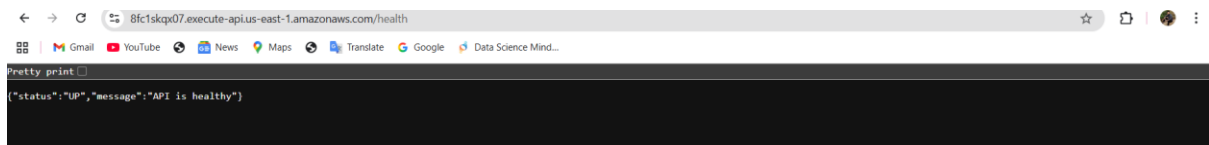
Selected default in stage name



After creating API I have selected default in left side and copied link invoke url



paste it in browser and it is showing healthy.



API Gateway Task 2: Secure API Access

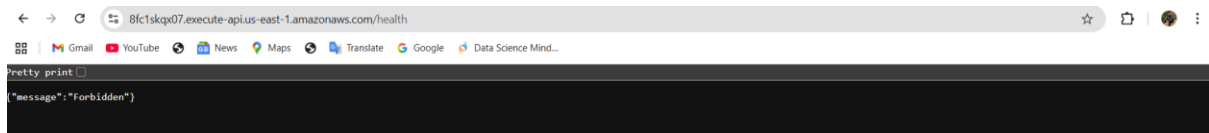
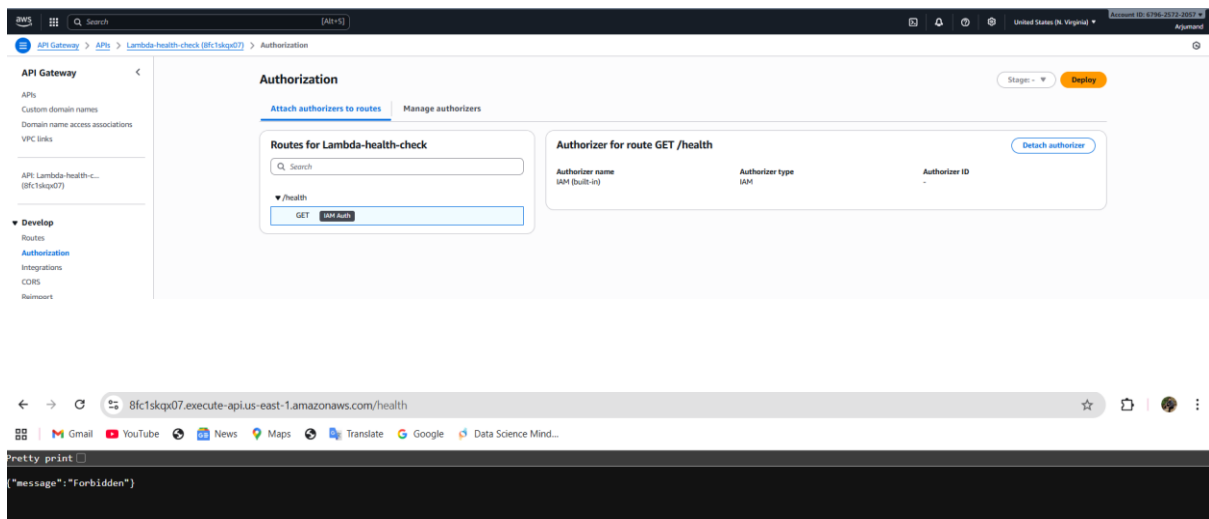
Goal: Allow only authorized users.

What to do

1. Enable IAM authentication
2. Restrict API to specific IAM user/role
3. Test authorized vs unauthorized request

You will learn

- API security
- IAM permissions



```
ArjumandM@Arjumand MINGW64 ~ (master)
$ aws configure
AWS Access Key ID [*****]: AKIAZ4PGZBDE7G4WC2W
AWS Secret Access Key [*****]: 90YBQRJuaQp0DesX7Uawxr3lwH6jUCHI+OK2oG7
Default region name [us-east-1]:
Default output format [json]:

ArjumandM@Arjumand MINGW64 ~ (master)
$ aws s3 ls
2025-12-17 15:29:21 aws-landa-mybucket1

ArjumandM@Arjumand MINGW64 ~ (master)
$ aws apigatewayv2 invoke-api \
> --api-id 8fclskqx07 \
> --stage $default \
> --method GET \
> --path '/health'

aws.exe: [ERROR]: argument operation: Found invalid choice 'invoke-api'

usage: aws [options] <command> [<subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:

    aws help
    aws <command> help
    aws <command> <subcommand> help

ArjumandM@Arjumand MINGW64 ~ (master)
$ pip install awscli
collecting awscli
  Downloading awscli-0.36-py3-none-any.whl.metadata (384 bytes)
```

```
ArjumandM@Arjumand MINGW64 ~ (master)
$ zonawscli \
> --service execute-api \
> --region us-east-1 \
> https://8fclskqx07.execute-api.us-east-1.amazonaws.com/health
{"status": "UP", "message": "API is healthy"}
```